



unc



FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Sistemas de Computación

TRABAJO PRÁCTICO N° 5:

Device Drivers

Alumnos:

García, Lautaro Misael

Gomez, Dolores

Renaudo Gaggioli, Valentino

Introducción

Los sistemas operativos modernos abstraen el acceso a dispositivos físicos mediante drivers, componentes de software que residen en espacio de kernel y exponen una interfaz uniforme (open, read, write, close) a las aplicaciones de usuario. En Linux, esta abstracción se sostiene sobre el principio "everything is a file": cualquier periférico puede representarse como un nodo en /dev, independientemente de su complejidad interna.

Este trabajo práctico aborda el diseño e implementación de un Character Device Driver (CDD) capaz de sensar dos señales externas con un período de muestreo de 1 segundo, permitiendo a una aplicación de usuario seleccionar el canal a leer, graficarlo en tiempo real y resetear la visualización al conmutar de señal. Se adoptó un enfoque de compilación cruzada (cross-compilation), desarrollando en una estación de trabajo Ubuntu y desplegando los binarios sobre una Raspberry Pi Zero W mediante SSH.

Marco Teórico

Drivers y abstracción de hardware. Un driver actúa como capa de traducción entre las llamadas al sistema del espacio de usuario y los registros o recursos físicos del hardware. Esto permite portabilidad de las aplicaciones y centraliza la lógica de control de bajo nivel dentro del kernel. En Linux, los drivers implementan interfaces estandarizadas que permiten a las aplicaciones interactuar con dispositivos sin necesidad de conocer detalles específicos de la plataforma.

Dispositivos de caracteres en Linux. Linux distingue entre block devices y character devices. Los character devices transmiten información como flujo secuencial de bytes, sin operaciones de seek ni cacheo de bloques, lo que se ajusta naturalmente a sensores y fuentes de datos en tiempo real. Se exponen como nodos especiales dentro del directorio /dev y son accedidos mediante operaciones estándar como open(), read(), write() y close().

Separación entre kernel space y user space. El kernel (Ring 0) posee acceso irrestricto a memoria y recursos de hardware, mientras que los procesos de usuario (Ring 3) se ejecutan en espacios virtuales aislados mediante la MMU. Esta separación mejora la estabilidad y seguridad del sistema operativo. El intercambio de datos entre ambos contextos requiere funciones específicas como:

```
long copy_to_user(void __user *to, const void *from, unsigned long n);  
long copy_from_user(void *to, const void __user *from, unsigned long n);
```

Estas funciones verifican la validez de las direcciones involucradas y previenen accesos indebidos a memoria.

GPIO (General Purpose Input/Output). Los GPIO son pines digitales de propósito general que permiten a los sistemas embebidos interactuar con dispositivos externos. En Raspberry Pi estos pines pueden configurarse como entradas o salidas mediante los controladores proporcionados por el kernel Linux. Los GPIO constituyen una de las interfaces más utilizadas para la adquisición de señales digitales provenientes de sensores, pulsadores o sistemas externos.

GPIO Descriptor Consumer Interface (gpiod). Las versiones modernas del kernel Linux utilizan una capa de abstracción basada en descriptores GPIO. Esta interfaz reemplaza mecanismos más antiguos basados en números globales de GPIO y proporciona una forma más portable y mantenible de acceder a los pines digitales. Funciones como `gpio_to_desc()` y `gpiod_get_value()` permiten obtener descriptores asociados a cada GPIO y consultar su estado lógico desde el espacio de kernel.

Compilación cruzada (Cross Compilation). La compilación cruzada consiste en generar ejecutables destinados a una arquitectura diferente de aquella donde se realiza la compilación. En este trabajo se utilizó un sistema Ubuntu x86_64 para producir módulos destinados a una

Raspberry Pi Zero W basada en arquitectura ARM. Para ello se empleó una toolchain ARM junto con los encabezados y archivos de construcción correspondientes al kernel ejecutado en la placa. Esta técnica permite reducir significativamente los tiempos de desarrollo y compilación respecto a realizar el proceso directamente sobre el sistema embebido.

Interfaces Web para Sistemas Embebidos. Una alternativa eficiente para visualizar datos provenientes de sistemas embebidos consiste en utilizar aplicaciones web livianas. Mediante un servidor HTTP ejecutándose localmente sobre la Raspberry Pi, los datos obtenidos por el driver pueden publicarse y visualizarse desde cualquier navegador conectado a la misma red. Esta arquitectura evita la necesidad de instalar aplicaciones específicas en los equipos clientes y facilita el acceso remoto a la información adquirida.

Arquitectura de tres capas

La arquitectura de tres capas es un modelo de diseño de software que divide un sistema en componentes independientes según sus funciones. Su principal objetivo es separar responsabilidades, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

Esta arquitectura se compone de tres niveles:

- **Capa de Hardware o Acceso a Datos:** se encarga de la comunicación directa con los dispositivos físicos o recursos del sistema. Generalmente está implementada mediante controladores de dispositivos (drivers) que abstraen las características del hardware.
- **Capa de Lógica de Aplicación:** procesa la información recibida, implementa las reglas de funcionamiento del sistema y coordina la interacción entre el hardware y la interfaz de usuario.
- **Capa de Presentación:** proporciona la interfaz con la que interactúa el usuario, mostrando información y permitiendo realizar configuraciones o enviar comandos al sistema.

La principal ventaja de esta arquitectura es que cada capa puede modificarse o actualizarse de manera independiente, reduciendo la complejidad del desarrollo y mejorando la organización general del sistema.

Desarrollo

1. Preparación del entorno de desarrollo. Se estableció conexión SSH con la Raspberry Pi Zero W (ssh pi@192.168.1.38), verificando kernel 6.12.75+rpt-rpi-v6, sistema Raspbian 12 (Bookworm) y arquitectura armv6l. Esta información es crítica para configurar correctamente la toolchain de compilación cruzada.

```
pi@rpizero: ~  
vale@ThinkBook-14:~$ ssh pi@rpizero.local  
The authenticity of host 'rpizero.local (192.168.1.38)' can't be established.  
ED25519 key fingerprint is SHA256:jlCzjuKgc2P9K8j+7Sz7D9FR2Fz0IIqQ6dZ9VTMQYp4.  
This host key is known by the following other names/addresses:  
  ~/.ssh/known_hosts:4: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'rpizero.local' (ED25519) to the list of known hosts.  
pi@rpizero.local's password:  
Linux rpizero 6.12.75+rpt-rpi-v6 #1 Raspbian 1:6.12.75-1+rpt1-bookworm (2026-03-11) armv6l  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Sun Jun  7 21:26:02 2026 from 192.168.1.74  
  
SSH is enabled and the default password for the 'pi' user has not been changed.  
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.  
  
pi@rpizero:~$ |
```

```
pi@rpizero: ~  
pi@rpizero:~$ uname -a  
Linux rpizero 6.12.75+rpt-rpi-v6 #1 Raspbian 1:6.12.75-1+rpt1-bookworm (2026-03-11) armv6l GNU/Linux  
pi@rpizero:~$ cat /etc/os-release  
PRETTY_NAME="Raspbian GNU/Linux 12 (bookworm)"  
NAME="Raspbian GNU/Linux"  
VERSION_ID="12"  
VERSION="12 (bookworm)"  
VERSION_CODENAME=bookworm  
ID=raspbian  
ID_LIKE=debian  
HOME_URL="http://www.raspbian.org/"  
SUPPORT_URL="http://www.raspbian.org/RaspbianForums"  
BUG_REPORT_URL="http://www.raspbian.org/RaspbianBugs"  
pi@rpizero:~$ |
```

2. Verificación del entorno para módulos del kernel. Se confirmó la presencia de linux-headers-6.12.75+rpt-rpi-v6 y el árbol /lib/modules/6.12.75+rpt-rpi-v6/build, junto con GCC 12.2.0 y GNU Make 4.3, validando que la plataforma está lista para compilar módulos externos sin instalaciones adicionales.

```
pi@rpizero:~$ ls /lib/modules/$(uname -r)/build
arch include Makefile Module.symvers scripts tools
pi@rpizero:~$ dpkg -l | grep raspberry-kernel
ii raspberry-kernel 1:1.20230405-1 armhf Header files for the Raspberry Pi Linux kernel
pi@rpizero:~$ dpkg -l | grep headers
ii libpigpio-dev 1.79-1+rpt1 armhf Development headers for client libraries for Raspberry Pi GPIO control
ii linux-headers-6.12.75+rpt-common-rpi 1:6.12.75-1+rpt1-bookworm all Common header files for Linux 6.12.75+rpt-rpi
ii linux-headers-6.12.75+rpt-rpi-v6 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux 6.12.75+rpt-rpi-v6
ii linux-headers-6.12.75+rpt-rpi-v7 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux 6.12.75+rpt-rpi-v7
ii linux-headers-6.12.75+rpt-rpi-v7l 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux 6.12.75+rpt-rpi-v7l
ii linux-headers-rpi-v6 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux rpi-v6 configuration (meta-package)
ii linux-headers-rpi-v7 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux rpi-v7 configuration (meta-package)
ii linux-headers-rpi-v7l 1:6.12.75-1+rpt1-bookworm armhf Header files for Linux rpi-v7l configuration (meta-package)
ii linux-libc-dev 1:6.12.75-1+rpt1-bookworm all Linux support headers for userspace development
ii raspberry-kernel-headers 1:1.20230405-1 armhf Header files for the Raspberry Pi Linux kernel
pi@rpizero:~$
```

3. Primer módulo del kernel. Se desarrolló un módulo mínimo (hello.c + Makefile), transferido vía scp, con funciones de inicialización y finalización que registran mensajes en el log del kernel. Se validó el ciclo completo: compilación → `insmod` → `dmesg` → `rmod` → `dmesg`, confirmando el flujo básico de desarrollo de LKMs.

```
pi@rpizero:~/tp5/driver/hello$ make
make -C /lib/modules/6.12.75+rpt-rpi-v6/build M=/home/pi/tp5/driver/hello modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
CC [M] /home/pi/tp5/driver/hello/hello.o
MODPOST /home/pi/tp5/driver/hello/Module.symvers
CC [M] /home/pi/tp5/driver/hello/hello.mod.o
CC [M] /home/pi/tp5/driver/hello/.module.common.o
LD [M] /home/pi/tp5/driver/hello/hello.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
pi@rpizero:~/tp5/driver/hello$ ls
hello.c hello.ko hello.mod hello.mod.c hello.mod.o hello.o Makefile modules.order Module.symvers
pi@rpizero:~/tp5/driver/hello$ sudo insmod hello.ko
pi@rpizero:~/tp5/driver/hello$ dmesg | tail
[ 48.691034] brcmfmac: brcmf_c_process_txcap_blob: no txcap_blob available (err=-2)
[ 48.692821] brcmfmac: brcmf_c_preinit_dcnds: Firmware: BCM43430/1 wl0: Jul 19 2021 03:24:18 version 7.45.98 (TOB) (56df937 CY) FWID 01-8e14b897
[ 58.244380] brcmfmac: brcmf_cfg80211_set_power_mgmt: power save enabled
[ 58.324165] Bluetooth: BNEP (Ethernet Emulation) ver 1.3
[ 58.324211] Bluetooth: BNEP filters: protocol multicast
[ 58.324244] Bluetooth: BNEP socket layer initialized
[ 58.476173] Bluetooth: MGMT ver 1.23
[ 58.649557] NET: Registered PF_ALG protocol family
[ 5091.862829] hello: loading out-of-tree module taints kernel.
[ 5091.863314] TP5: modulo hello cargado
pi@rpizero:~/tp5/driver/hello$ sudo rmod hello
pi@rpizero:~/tp5/driver/hello$ dmesg | tail
[ 48.692821] brcmfmac: brcmf_c_preinit_dcnds: Firmware: BCM43430/1 wl0: Jul 19 2021 03:24:18 version 7.45.98 (TOB) (56df937 CY) FWID 01-8e14b897
[ 58.244380] brcmfmac: brcmf_cfg80211_set_power_mgmt: power save enabled
[ 58.324165] Bluetooth: BNEP (Ethernet Emulation) ver 1.3
[ 58.324211] Bluetooth: BNEP filters: protocol multicast
[ 58.324244] Bluetooth: BNEP socket layer initialized
[ 58.476173] Bluetooth: MGMT ver 1.23
[ 58.649557] NET: Registered PF_ALG protocol family
[ 5091.862829] hello: loading out-of-tree module taints kernel.
[ 5091.863314] TP5: modulo hello cargado
[ 5158.899027] TP5: modulo hello descargado
pi@rpizero:~/tp5/driver/hello$
```

4. Character Device Driver básico. Tomando como base el ejemplo `drv3`, se renombraron estructuras y funciones al contexto del proyecto (`tp5_init`, `tp5_open`, `tp5_release`, `tp5_fops`), se definieron macros `DEVICE_NAME` y `CLASS_NAME`, se uniformaron los mensajes de `printk()` y se reorganizó el código en secciones (operaciones, `file_operations`, `init`, `exit`). Se detectó y corrigió una incompatibilidad de API: en Linux 6.12 `class_create()` recibe un solo argumento (el nombre), a diferencia de versiones previas que requerían `THIS_MODULE`.

```
pi@rpizero: ~/tp5/driver/cdd_basico
pi@rpizero:~/tp5/driver/cdd_basico $ ls
cdd_basico.c Makefile
pi@rpizero:~/tp5/driver/cdd_basico $ nano cdd_basico.c
pi@rpizero:~/tp5/driver/cdd_basico $ make
make -C /lib/modules/6.12.75+rpt-rpi-v6/build M=/home/pi/tp5/driver/cdd_basico modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
  CC [M] /home/pi/tp5/driver/cdd_basico/cdd_basico.o
  MODPOST /home/pi/tp5/driver/cdd_basico/Module.symvers
  CC [M] /home/pi/tp5/driver/cdd_basico/cdd_basico.mod.o
  CC [M] /home/pi/tp5/driver/cdd_basico/.module-common.o
  LD [M] /home/pi/tp5/driver/cdd_basico/cdd_basico.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
pi@rpizero:~/tp5/driver/cdd_basico $ sudo insmod cdd_basico.ko
pi@rpizero:~/tp5/driver/cdd_basico $ ls -l /dev/tp5
crw----- 1 root root 239, 0 Jun  8 02:47 /dev/tp5
pi@rpizero:~/tp5/driver/cdd_basico $ cat /proc/devices | grep tp5
239 tp5
pi@rpizero:~/tp5/driver/cdd_basico $ echo hola > /dev/tp5
-bash: /dev/tp5: Permission denied
pi@rpizero:~/tp5/driver/cdd_basico $ sudo sh -c 'echo hola > /dev/tp5'
pi@rpizero:~/tp5/driver/cdd_basico $ cat /dev/tp5
cat: /dev/tp5: Permission denied
pi@rpizero:~/tp5/driver/cdd_basico $ sudo cat /dev/tp5
pi@rpizero:~/tp5/driver/cdd_basico $ dmesg | tail -n 20
[ 35.207214] [drm] Initialized vc4 0.0.0 for soc:gpu on minor 0
[ 35.217851] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
[ 35.220192] vc4-drm soc:gpu: [drm] Cannot find any crtc or sizes
[ 35.423832] brcmfmac: brcmf_c_process_txcap_blob: no txcap_blob available (err=-2)
[ 35.431795] brcmfmac: brcmf_c_preinit_dcmds: Firmware: BCM43430/1 wl0: Jul 19 2021 03:24:18 version 7.45.98 (
TOB) (56df937 CY) FWID 01-8e14b897
[ 45.263915] brcmfmac: brcmf_cfg80211_set_power_mgmt: power save enabled
[ 45.411489] Bluetooth: BNEP (Ethernet Emulation) ver 1.3
[ 45.411534] Bluetooth: BNEP filters: protocol multicast
[ 45.411569] Bluetooth: BNEP socket layer initialized
[ 45.606984] Bluetooth: MGMT ver 1.23
[ 45.760240] NET: Registered PF_ALG protocol family
[ 6228.600681] cdd_basico: loading out-of-tree module taints kernel.
[ 6228.601247] TP5: registrando character device
[ 6228.603511] TP5: device creado correctamente
[ 6370.759689] TP5: open()
[ 6370.759803] TP5: write()
[ 6370.759832] TP5: release()
[ 6441.802803] TP5: open()
[ 6441.802978] TP5: read()
[ 6441.803092] TP5: release()
pi@rpizero:~/tp5/driver/cdd_basico $
```

5. Intercambio copy_to_user / copy_from_user. Se implementó una variable interna de almacenamiento temporal: `write()` copia un byte desde espacio de usuario con `copy_from_user()`, y `read()` lo devuelve con `copy_to_user()`. Se validó con `echo A > /dev/tp5` y `cat /dev/tp5`. Este mecanismo es la base del protocolo de selección de canal: `echo 1 > /dev/tp5` / `echo 2 > /dev/tp5` para elegir señal, y `cat /dev/tp5` para leer el valor del canal activo.

```
pi@rpizero: ~/tp5/driver/cdd_loopback
pi@rpizero:~/tp5/driver/cdd_loopback $ make
make -C /lib/modules/6.12.75+rpt-rpi-v6/build M=/home/pi/tp5/driver/cdd_loopback modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
  CC [M]  /home/pi/tp5/driver/cdd_loopback/cdd_loopback.o
  MODPOST /home/pi/tp5/driver/cdd_loopback/Module.symvers
  CC [M]  /home/pi/tp5/driver/cdd_loopback/cdd_loopback.mod.o
  CC [M]  /home/pi/tp5/driver/cdd_loopback/.module-common.o
  LD [M]  /home/pi/tp5/driver/cdd_loopback/cdd_loopback.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v6'
pi@rpizero:~/tp5/driver/cdd_loopback $ sudo insmod cdd_loopback.ko
pi@rpizero:~/tp5/driver/cdd_loopback $ sudo sh -c 'echo A > /dev/tp5'
pi@rpizero:~/tp5/driver/cdd_loopback $ sudo cat /dev/tp5
Api@rpizero:~/tp5/driver/cdd_loopback $ sudo sh -c 'echo A\n > /dev/tp5'
pi@rpizero:~/tp5/driver/cdd_loopback $ sudo cat /dev/tp5
Api@rpizero:~/tp5/driver/cdd_loopback $ dmesg | tail -n 20
[ 6441.802803] TP5: open()
[ 6441.802978] TP5: read()
[ 6441.803092] TP5: release()
[ 6575.454852] TP5: device eliminado
[ 7584.409711] TP5: registrando character device
[ 7584.421067] TP5: device creado correctamente
[ 7599.056079] TP5: open()
[ 7599.056195] TP5: write()
[ 7599.056225] TP5: release()
[ 7608.035619] TP5: open()
[ 7608.035790] TP5: read()
[ 7608.037183] TP5: read()
[ 7608.037361] TP5: release()
[ 7639.377301] TP5: open()
[ 7639.377415] TP5: write()
[ 7639.377445] TP5: release()
[ 7641.622542] TP5: open()
[ 7641.622711] TP5: read()
[ 7641.624049] TP5: read()
[ 7641.624228] TP5: release()
```

6. Exploración del subsistema GPIO. Mediante `/sys/kernel/debug/gpio` se identificó el controlador `pinctrl-bcm2835` y se seleccionaron GPIO17 (gpio-529) y GPIO27 (gpio-539) como pines de entrada para las dos señales externas, por no interferir con periféricos del sistema.

7. Compilación cruzada. Se instaló la toolchain `gcc-arm-linux-gnueabi` / `binutils-arm-linux-gnueabi`. Una primera prueba con `hello-world.c` compiló correctamente como binario ARM de 32 bits, pero produjo *Segmentation Fault* en la Pi Zero W: `readelf -A` reveló que el binario se generaba para ARMv7, incompatible con el procesador ARMv6 de la Pi Zero. Se verificó que la compilación nativa en la Pi (gcc local) funciona sin problemas.

Para los módulos del kernel, se exportaron desde la Pi los headers (`linux-headers-6.12.75+rpt-rpi-v6` y `linux-headers-6.12.75+rpt-common-rpi`) y el árbol `linux-kbuild`. Se detectó que herramientas auxiliares de Kbuild (`fixdep`, `modpost`, `recordmcount`), exportadas como binarios ARM, producían *Exec format error* al ejecutarse en el host `x86_64`.

Conclusión: el compilador cruzado y los headers eran correctos, pero las herramientas de build deben generarse nativamente para el host; la estrategia recomendada es usar un árbol completo de fuentes del kernel Raspberry Pi en el host para que Kbuild regenere esas herramientas para `x86_64` mientras compila el módulo para ARM.

```
pi@rpizero: ~/tp5
vale@ThinkBook-14: ~/repos/SdC-TP-2026/TP5/user/arm_toolchain_test$ ls
hello-world.c  Makefile
vale@ThinkBook-14: ~/repos/SdC-TP-2026/TP5/user/arm_toolchain_test$ make
arm-linux-gnueabi-hf-gcc hello-world.c -o hello-world
vale@ThinkBook-14: ~/repos/SdC-TP-2026/TP5/user/arm_toolchain_test$ scp hello-world pi@rpizero.local:~/tp5
pi@rpizero.local's password:
hello-world                                100% 7816    37.2KB/s   00:00
vale@ThinkBook-14: ~/repos/SdC-TP-2026/TP5/user/arm_toolchain_test$ ssh pi@rpizero.local
pi@rpizero.local's password:
Permission denied, please try again.
pi@rpizero.local's password:
Linux rpizero 6.12.75+rpt-rpi-v6 #1 Raspbian 1:6.12.75-1+rpt1-bookworm (2026-03-11) armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Jun 11 13:30:38 2026 from 192.168.1.74

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@rpizero:~$ cd tp5
pi@rpizero:~/tp5$ ls
arm_toolchain_test  driver  hello-world
pi@rpizero:~/tp5$ ./hello-world
Segmentation fault
pi@rpizero:~/tp5$ readelf -A hello-world
Attribute Section: aeabi
File Attributes
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
  Tag_ARM_ISA_use: Yes
  Tag_THUMB_ISA_use: Thumb-2
  Tag_FP_arch: VFPv3-D16
  Tag_ABI_PCS_wchar_t: 4
  Tag_ABI_FP_rounding: Needed
  Tag_ABI_FP_denormal: Needed
  Tag_ABI_FP_exceptions: Needed
  Tag_ABI_FP_number_model: IEEE 754
  Tag_ABI_align_needed: 8-byte
  Tag_ABI_align_preserved: 8-byte, except leaf SP
  Tag_ABI_enum_size: int
  Tag_ABI_VFP_args: VFP registers
  Tag_CPU_unaligned_access: v6
pi@rpizero:~/tp5$
```

```
pi@rpizero: ~/tp5
pi@rpizero:~/tp5 $ cat > hello.c << 'EOF'
#include <stdio.h>

int main(void)
{
    printf("Hello, Arm World\n");
    return 0;
}
EOF
pi@rpizero:~/tp5 $ gcc hello.c -o hello
pi@rpizero:~/tp5 $ ./hello
Hello, Arm World
pi@rpizero:~/tp5 $ uname -m
armv6l
pi@rpizero:~/tp5 $ gcc -dumpmachine
arm-linux-gnueabi
pi@rpizero:~/tp5 $
```

```
vale@ThinkBook-14: ~/repos/SdC-TP-2026/TP5/toolchain
vale@ThinkBook-14:~/repos/SdC-TP-2026/TP5$ mkdir -p ~/repos/SdC-TP-2026/TP5/toolchain

scp pi@rpizero.local:~/rpi-kernel-headers.tar.gz \
~/repos/SdC-TP-2026/TP5/toolchain/
pi@rpizero.local's password:
rpi-kernel-headers.tar.gz
100% 10MB 28.6KB/s 06:15
vale@ThinkBook-14:~/repos/SdC-TP-2026/TP5$ cd toolchain/
vale@ThinkBook-14:~/repos/SdC-TP-2026/TP5/toolchain$ tar xzf rpi-kernel-headers.tar.gz
vale@ThinkBook-14:~/repos/SdC-TP-2026/TP5/toolchain$ tree -L 2
.
├── kernel-export
│   ├── linux-headers-6.12.75+rpt-common-rpi
│   └── linux-headers-6.12.75+rpt-rpi-v6
└── rpi-kernel-headers.tar.gz

vale@ThinkBook-14:~/repos/SdC-TP-2026/TP5/toolchain$

pi@rpizero:~
pi@rpizero:~ $ mkdir -p ~/kernel-export
pi@rpizero:~ $ cp -a /usr/src/linux-headers-6.12.75+rpt-rpi-v6 ~/kernel-export/
cp -a /usr/src/linux-headers-6.12.75+rpt-common-rpi ~/kernel-export/

tar czf rpi-kernel-headers.tar.gz kernel-export
^C
pi@rpizero:~ $ ls
kernel-export  SignalMux  tp5
pi@rpizero:~ $ ls ./kernel-export/
linux-headers-6.12.75+rpt-common-rpi  linux-headers-6.12.75+rpt-rpi-v6
pi@rpizero:~ $ tar czf rpi-kernel-headers.tar.gz kernel-export
pi@rpizero:~ $ ls
kernel-export  rpi-kernel-headers.tar.gz  SignalMux  tp5
pi@rpizero:~ $ ls ./kernel-export/
linux-headers-6.12.75+rpt-common-rpi  linux-headers-6.12.75+rpt-rpi-v6
pi@rpizero:~ $ ls -lh rpi-kernel-headers.tar.gz
-rw-r--r-- 1 pi pi 11M Jun 11 19:34 rpi-kernel-headers.tar.gz
pi@rpizero:~ $
```

Implementación del driver final: adquisición de señales GPIO

Una vez validados los conceptos básicos de character devices, intercambio de datos con `copy_to_user()` y `copy_from_user()`, y compilación cruzada para ARM, se desarrolló la versión final del módulo `cdd_gpio`.

A diferencia del prototipo inicial basado en señales generadas por software mediante temporizadores del kernel, la versión final cumple estrictamente con la consigna del trabajo práctico, realizando la adquisición de señales externas reales conectadas a los GPIO de la Raspberry Pi Zero W.

Selección de GPIOs

Se utilizaron los pines GPIO17 y GPIO27 como entradas digitales. Ambos pines fueron identificados mediante el subsistema GPIO del kernel Linux y seleccionados por no interferir con periféricos críticos de la plataforma.

Acceso mediante gpiod

El driver obtiene los descriptores GPIO utilizando la API moderna del kernel:

`gpio_to_desc()`

Esta interfaz reemplaza mecanismos más antiguos y permite acceder al estado lógico de los pines de manera independiente de la implementación interna del controlador GPIO.

Protocolo de selección de canal

La operación `write()` interpreta un único carácter recibido desde espacio de usuario:

'1' → selecciona GPIO17

'2' → selecciona GPIO27

El canal seleccionado se almacena internamente dentro del driver.

Lectura de señales

La operación `read()` consulta el estado lógico del GPIO actualmente seleccionado mediante:

`gpiod_get_value()`

El resultado se convierte a texto ASCII ("0" o "1") y se transfiere al espacio de usuario utilizando `copy_to_user()`.

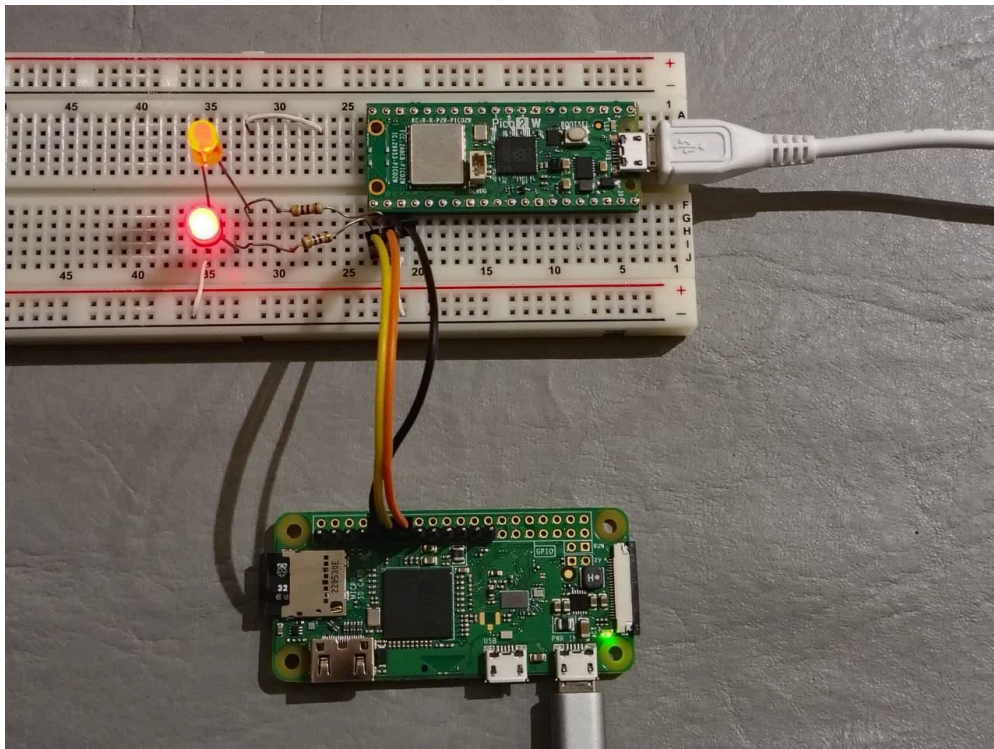
De esta forma, las aplicaciones no acceden directamente al hardware, sino a través de la interfaz estándar proporcionada por el character device `/dev/tp5`.

Validación sobre hardware real

Para validar el funcionamiento del sistema se utilizaron dos señales digitales externas generadas por una Raspberry Pi Pico.

La primera señal fue configurada con período de 1 segundo, mientras que la segunda utilizó un período de 250 ms. Ambas señales fueron conectadas a la Raspberry Pi Zero W mediante GPIO17 y GPIO27 respectivamente.

Adicionalmente se incorporaron LEDs indicadores conectados a las líneas de señal para facilitar la verificación visual del estado lógico presente en cada entrada.



Pruebas desde consola

Se verificó el correcto funcionamiento del driver utilizando herramientas estándar de Linux:

```
echo -n "1" > /dev/tp5
```

```
cat /dev/tp5
```

```
echo -n "2" > /dev/tp5
```

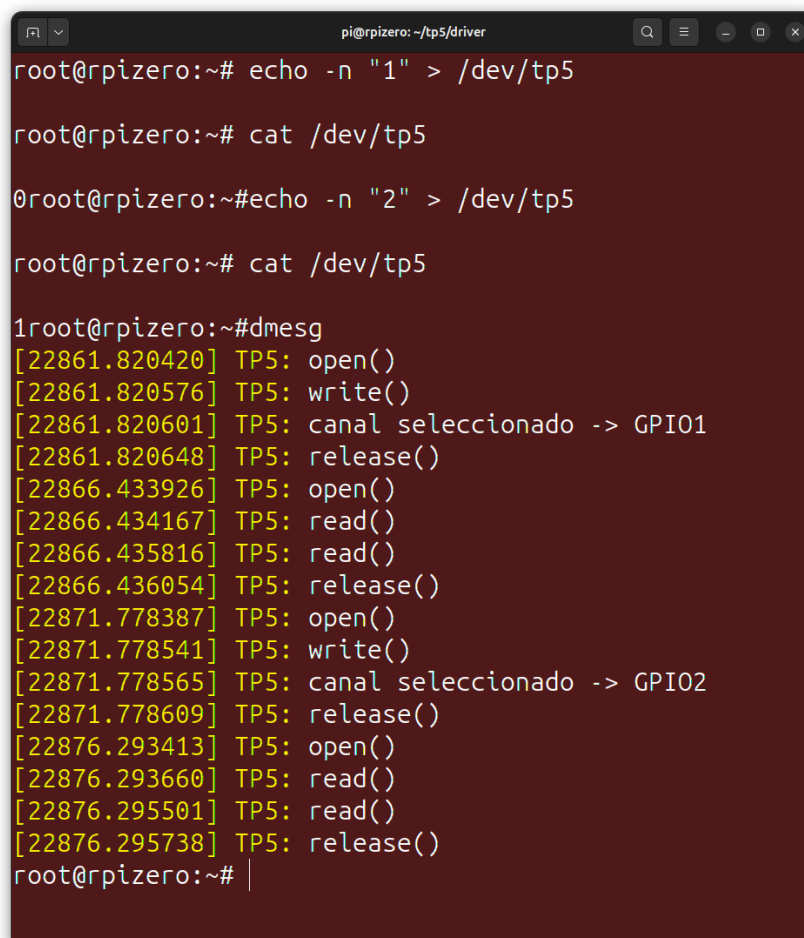
```
cat /dev/tp5
```

Los resultados obtenidos coincidieron con los estados eléctricos aplicados sobre cada GPIO.

Verificación mediante mensajes del kernel

El registro del kernel (dmesg) confirmó la ejecución correcta de las operaciones `open()`, `read()`, `write()` y `release()`, así como la conmutación entre ambos canales de adquisición.

Esto permitió comprobar el correcto funcionamiento de la interfaz entre espacio de usuario y espacio de kernel.



```
pi@rpizero: ~/tp5/driver
root@rpizero:~# echo -n "1" > /dev/tp5
root@rpizero:~# cat /dev/tp5
0root@rpizero:~#echo -n "2" > /dev/tp5
root@rpizero:~# cat /dev/tp5
1root@rpizero:~#dmesg
[22861.820420] TP5: open()
[22861.820576] TP5: write()
[22861.820601] TP5: canal seleccionado -> GPIO1
[22861.820648] TP5: release()
[22866.433926] TP5: open()
[22866.434167] TP5: read()
[22866.435816] TP5: read()
[22866.436054] TP5: release()
[22871.778387] TP5: open()
[22871.778541] TP5: write()
[22871.778565] TP5: canal seleccionado -> GPIO2
[22871.778609] TP5: release()
[22876.293413] TP5: open()
[22876.293660] TP5: read()
[22876.295501] TP5: read()
[22876.295738] TP5: release()
root@rpizero:~# |
```

Aplicación de usuario

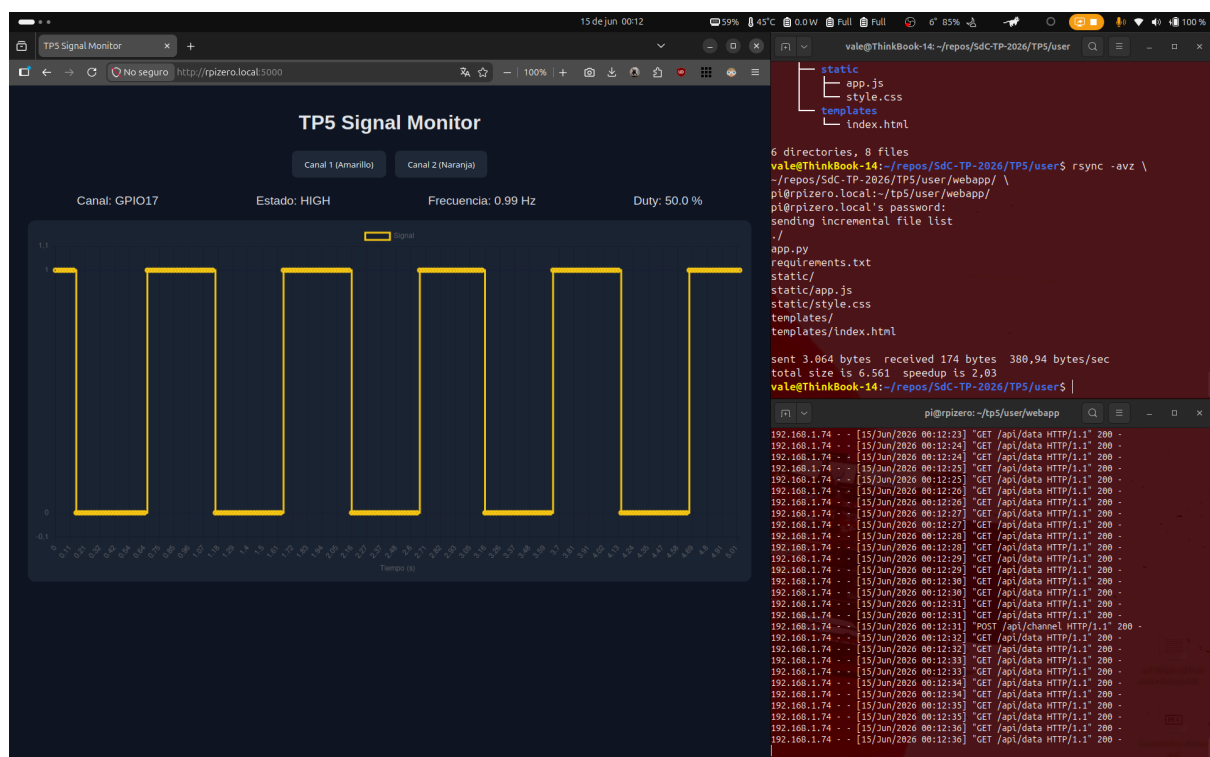
Se desarrollaron dos herramientas de usuario:

- Una aplicación de línea de comandos (monitor.py), capaz de adquirir muestras periódicas desde /dev/tp5, calcular frecuencia y duty cycle, y mostrar la información en tiempo real.
- Una interfaz web desarrollada con Flask y Chart.js, accesible desde cualquier navegador conectado a la misma red.

La interfaz web permite:

- Seleccionar el canal de adquisición mediante botones.
- Visualizar la señal en tiempo real mediante un gráfico desplazable.
- Reiniciar automáticamente el gráfico al cambiar de canal.
- Mostrar frecuencia estimada y duty cycle calculados a partir de las muestras adquiridas.
- Visualizar la evolución temporal utilizando un eje horizontal expresado en segundos.

Las pruebas realizadas permitieron observar correctamente tanto la señal de 1 segundo como la señal de 250 ms generadas por la Raspberry Pi Pico.



Resolución de la compilación cruzada para ARM

Durante la compilación cruzada del driver final se presentó un obstáculo relacionado con las herramientas auxiliares del sistema de construcción Kbuild —**fixdep**, **recordmcount** y **modpost**—. Estas herramientas no forman parte del binario final (el **.ko** para ARM), sino que son ejecutadas por el propio proceso de build en la máquina host durante la compilación, cumpliendo tareas de análisis de dependencias, instrumentación y verificación de símbolos del kernel.

Dado que dichas herramientas habían sido obtenidas exportando directamente los archivos **scripts/** y **tools/** desde la Raspberry Pi, se encontraban compiladas para arquitectura ARM. Al ejecutarse sobre el host x86_64 durante la compilación cruzada, el sistema operativo no

podía interpretar dichos binarios, produciendo el error **Exec format error**. Esto evidenció la distinción fundamental entre los binarios que deben generarse para la arquitectura destino (el módulo del kernel) y las herramientas que deben ejecutarse de forma nativa en el host.

```
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-common-rpi$ cd ~/tp5/driver/cdd_signals_arm
make
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -C /home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-rpi-v6 M=/home/dolores/tp5/driver/cdd_signals_arm modules
make[1]: se entra en el directorio '/home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-rpi-v6'
CC [M] /home/dolores/tp5/driver/cdd_signals_arm/cdd_signals.o
/bin/sh: 1: scripts/basic/fixdep: Exec format error
make[3]: *** [/home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-common-rpi/scripts/Makefile.build:249: /home/dolores/tp5/driver/cdd_signals_arm/cdd_signals.o] Error 126
make[3]: *** Se borra el archivo '/home/dolores/tp5/driver/cdd_signals_arm/cdd_signals.o'
make[2]: *** [/home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-common-rpi/Makefile:197: /home/dolores/tp5/driver/cdd_signals_arm] Error 2
make[1]: *** [/usr/src/linux-headers-6.12.75+rpt-common-rpi/Makefile:236: __sub-make] Error 2
make[1]: se sale del directorio '/home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-rpi-v6'
make: *** [Makefile:7: all] Error 2
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$ file ~/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-kbuild-6.12.75+rpt/scripts/basic/fixdep
/home/dolores/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-kbuild-6.12.75+rpt/scripts/basic/fixdep: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, BuildID[sha1]=dfabe2fc8654e15d336826a64e1ccd6764704d6d, for GNU/Linux 3.2.0, stripped
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$
```

```
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-kbuild-6.12.75+rpt/scripts/mad$ cd ~/tp5/toolchain
git clone --depth=1 --branch rpi-6.12.y https://github.com/raspberrypi/linux.git rpi-linux
Clonando en 'rpi-linux'...
remote: Enumerating objects: 92659, done.
remote: Counting objects: 100% (92659/92659), done.
remote: Compressing objects: 100% (82577/82577), done.
remote: Total 92659 (delta 9540), reused 82004 (delta 9096), pack-reused 0 (from 0)
Recibiendo objetos: 100% (92659/92659), 258.39 MiB | 10.81 MiB/s, listo.
Resolviendo deltas: 100% (9540/9540), listo.
Actualizando archivos: 100% (87439/87439), listo.
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain$
```

La resolución consistió en clonar el árbol completo de fuentes del kernel Raspberry Pi (rama **rpi-6.12.y**) en el host, incorporar la configuración (**.config**) correspondiente a la versión de kernel instalada en la Raspberry Pi Zero W, y ejecutar **make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules_prepare**. Este comando regeneró automáticamente todas las herramientas auxiliares de Kbuild como binarios nativos **x86_64**, permitiendo completar la compilación cruzada del módulo sin necesidad de reconstruir manualmente cada herramienta.

Versión del compilador cruzado. El archivo `.kernelvariables` del kernel de la Pi fuerza el uso de `arm-linux-gnueabi-hf-gcc-12`, mientras que el toolchain disponible en el host instaló la versión 13. Se resolvió mediante un enlace simbólico `arm-linux-gnueabi-hf-gcc-12 -> arm-linux-gnueabi-hf-gcc-13`.

```
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain$ cp ~/tp5/toolchain/rpi-kernel-headers-v4/kernel-export-v2/linux-headers-6.12.75+rpt-rpi-v6/.config ~/tp5/toolchain/rpi-linux/.config
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain$ cd ~/tp5/toolchain/rpi-linux
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-hf- olddefconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
#
# configuration written to .config
#
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/toolchain/rpi-linux$
```

Herramientas auxiliares de Kbuild compiladas para ARM. Tras resolver los puntos anteriores, la compilación avanzó hasta el paso `CC [M]`, pero falló sucesivamente al intentar ejecutar `fixdep`, `recordmcount` y `modpost` —todos exportados como binarios ARM desde la Pi— sobre el host `x86_64` (`Exec format error`). Se intentó recompilar `fixdep` de forma nativa a partir de su código fuente (`fixdep.c`), lo cual requirió además reconstruir manualmente el header auxiliar `xalloc.h` (ausente en el árbol exportado), logrando obtener un binario `fixdep` `x86_64` funcional.

Símbolos del kernel. Se copió el archivo `Module.symvers` (exportado originalmente desde la Pi) al árbol clonado, necesario para que `modpost` resuelva los símbolos del kernel (`register_chrdev_region`, `class_create`, `mod_timer`, `_printk`, etc.) utilizados por el driver.

Resultado. La compilación cruzada finalizó exitosamente:

```
CC [M] cdd_signals.o
MODPOST cdd_signals.mod.o
LD [M] cdd_signals.ko
```

generando un módulo `cdd_signals.ko` para arquitectura ARM, verificado mediante `file` y `readelf -A` para confirmar su compatibilidad con el procesador ARMv6 de la Raspberry Pi Zero W.


```
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$ cd ~/tp5/driver/cdd_signals_arm
make
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -C /home/dolores/tp5/toolchain/rpi-linux M=/home/dolores/tp5/driver/cdd_signals_a
rm modules
make[1]: se entra en el directorio '/home/dolores/tp5/toolchain/rpi-linux'
  MODPOST /home/dolores/tp5/driver/cdd_signals_arm/Module.symvers
  CC [M] /home/dolores/tp5/driver/cdd_signals_arm/cdd_signals.mod.o
  CC [M] /home/dolores/tp5/driver/cdd_signals_arm/.module-common.o
  LD [M] /home/dolores/tp5/driver/cdd_signals_arm/cdd_signals.ko
make[1]: se sale del directorio '/home/dolores/tp5/toolchain/rpi-linux'
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$ ls -la cdd_signals.ko
file cdd_signals.ko
-rw-rw-r-- 1 dolores dolores 10140 jun 13 21:02 cdd_signals.ko
cdd_signals.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=cb0fedbbff5124ae418428bc68859c61b707e50f, n
ot stripped
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$
```

```
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$ arm-linux-gnueabihf-readelf -A cdd_signals.ko
Attribute Section: aeabi
Atributos de Fichero
  Tag_CPU_name: "6K"
  Tag_CPU_arch: v6K
  Tag_ARM_ISA_use: Yes
  Tag_THUMB_ISA_use: Thumb-1
  Tag_ABI_PCS_wchar_t: 2
  Tag_ABI_FP_denormal: Needed
  Tag_ABI_FP_exceptions: Needed
  Tag_ABI_FP_number_model: IEEE 754
  Tag_ABI_align_needed: 8 bytes
  Tag_ABI_align_preserved: 8 bytes, excepto SP leaf
  Tag_ABI_enum_size: int
  Tag_ABI_optimization_goals: Aggressive Speed
  Tag_CPU_unaligned_access: v6
(venv) dolores@dolores-IdeaPad-3-15IML05:~/tp5/driver/cdd_signals_arm$
```

Conclusión

En este trabajo práctico se desarrolló un sistema completo de adquisición y visualización de señales digitales utilizando una arquitectura de tres capas compuesta por hardware externo, un driver de kernel Linux y aplicaciones de usuario.

El módulo `cdd_gpio` implementado expone una interfaz estándar de tipo character device mediante `/dev/tp5`, permitiendo seleccionar y leer señales digitales provenientes de GPIO17 y GPIO27 sin que las aplicaciones deban acceder directamente al hardware.

La solución fue desplegada y validada sobre una Raspberry Pi Zero W ejecutando Linux 6.12, utilizando señales reales generadas por una Raspberry Pi Pico. Las pruebas realizadas permitieron verificar el correcto funcionamiento del driver, la comunicación entre espacio de usuario y espacio de kernel, y la adquisición de señales digitales externas.

Además del desarrollo del driver, se implementaron herramientas de monitoreo tanto por línea de comandos como mediante una interfaz web basada en Flask y Chart.js. Estas aplicaciones permitieron visualizar las señales en tiempo real y calcular parámetros relevantes como frecuencia y duty cycle.

Durante el proyecto también se abordaron problemas reales de desarrollo sobre sistemas embebidos Linux, incluyendo compilación cruzada para ARM, compatibilidad entre versiones del kernel, preparación del árbol de fuentes y despliegue remoto mediante SSH.

El resultado final cumple con los objetivos planteados por la consigna, integrando hardware, kernel Linux y aplicaciones de usuario en un sistema funcional de adquisición de señales digitales.